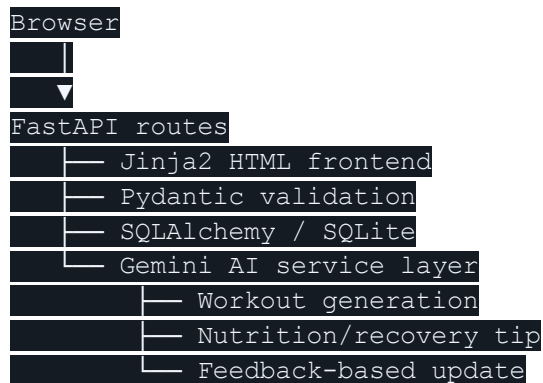


Features

- User form: name, user ID, age, weight, fitness goal and workout intensity.
- Gemini-powered 7-day structured workout plan.
- Gemini Flash nutrition/recovery tip.
- Feedback-based plan regeneration.
- SQLite persistence through SQLAlchemy.
- Admin dashboard showing users and original/updated plans.
- HTML frontend and JSON API.
- Pydantic validation.
- Automated tests that do not require a live Gemini request.
- Phase-wise project documentation matching the required 8 project phases.

Architecture



The supplied document refers to the older `google-generativeai` SDK and Gemini 1.5 Pro/Flash. This implementation uses Google's current `google-genai`

SDK and keeps model names configurable in `.env`, so the application code does not have to change when a model name changes.

Google's current documentation supports Pydantic-backed structured JSON output with the GenAI SDK:

<https://ai.google.dev/gemini-api/docs/structured-output>

Requirements

- Python 3.11+
- VS Code
- Gemini API key
- Internet connection for live Gemini generation

Quick start — Windows PowerShell

```
cd FitBuddy-AI-Fitness-Plan-Generator
```

```
py -3.11 -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

```
Copy-Item .env.example .env
```

Open `.env` and replace `YOUR_GEMINI_API_KEY` with your key.

Then:

```
uvicorn app.main:app --reload
```

Open:

- <http://127.0.0.1:8000>
- <http://127.0.0.1:8000/docs>
- http://127.0.0.1:8000/view-all-users?admin_key=fitbuddy-demo-admin

Windows CMD

```
python -3.11 -m venv .venv  
.venv\Scripts\activate  
python -m pip install --upgrade pip  
pip install -r requirements.txt  
copy .env.example .env
```

```
uvicorn app.main:app --reload
```

macOS / Linux

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip  
pip install -r requirements.txt  
cp .env.example .env
```

```
uvicorn app.main:app --reload
```

Environment configuration

`.env.example:`

```
GEMINI_API_KEY=replace_with_your_gemini_api_key
GEMINI_WORKOUT_MODEL=gemini-2.5-pro
GEMINI_TIP_MODEL=gemini-2.5-flash
GEMINI_UPDATE_MODEL=gemini-2.5-pro
DATABASE_URL=sqlite:///./fitbuddy.db
ADMIN_KEY=fitbuddy-demo-admin
APP_NAME=FitBuddy
```

```
DEBUG=true
```

Do not commit `.env`.

If a configured model is unavailable for your Gemini account, change only the model name in `.env` to a currently available Gemini model.

API

GET `/api/health`

Returns:

```
{"status": "ok", "service": "FitBuddy"}
```

```
POST /api/generate-workout
```

```
{  
  "username": "Demo User",  
  "user_id": "demo-001",  
  "age": 20,  
  "weight": 65,  
  "goal": "general wellness",  
  "intensity": "medium"
```

```
}
```

```
POST /api/submit-feedback
```

```
{  
  "user_id": "demo-001",  
  "feedback": "Add more mobility work and keep one recovery day."
```

```
}
```

Interactive API testing is available at </docs>.

Testing

With the virtual environment active:

```
pytest -q
```

Tests cover:

- application startup
- home page rendering
- health endpoint
- input validation
- database persistence
- admin access control
- SON API behaviour

The tests mock Gemini calls, so a Gemini API key is not required to run the automated test suite.

Project structure

```
FitBuddy-AI-Fitness-Plan-Generator/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── templates.py
│   ├── routes.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── result.html
│   ├── all_users.html
│   └── error.html
├── static/
│   ├── css/style.css
│   └── js/app.js
├── tests/
│   ├── conftest.py
│   └── test_app.py
├── docs/
│   ├── phase-1-brainstorming.md
│   ├── phase-2-requirements.md
│   ├── phase-3-design.md
│   └── phase-4-planning.md
```

```
| — phase-5-development.md
| — phase-6-testing.md
| — phase-7-documentation.md
| — phase-8-demonstration.md
| — data/
|   — .gitkeep
| — .env.example
| — .gitignore
| — requirements.txt
| — README.md
```

Demo flow

1. Start the application.
2. Fill the FitBuddy form.
3. Generate the 7-day plan.
4. Review the workout and nutrition/recovery tip.
5. Submit feedback.
6. Review the updated plan.
7. Open the admin dashboard.
8. Open `/docs` and demonstrate the JSON API.
9. Run `pytest -q`.

Safety

This is an educational wellness application, not a medical or clinical system. AI output should not be treated as diagnosis, treatment or individualized medical advice. The AI prompts intentionally avoid dangerous challenges, extreme dieting, medication/supplement prescriptions and unsafe training instructions.

